



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

T.Y B. Tech, Semester: VI

Academic Year 2024-25

Name: Gargi Sukhatankar

SAPID: 60019220125

Batch: B1

Roll No: B027

Experiment No. 9

Aim: To implement data cataloguing and Sharing Services by using open-source tools (Any)

Theory:

Data cataloging is the process of creating an organized inventory of data assets across an organization. A data catalog contains metadata — information about the data such as the data's source, structure, usage, owner, and lineage. It helps users discover, understand, and trust the data they use.

- **Purpose:** Simplifies data discovery.
- **Metadata Types:** Technical (e.g., schema, data types), business (e.g., glossary terms), and operational (e.g., usage logs, lineage).
- **Automated Cataloging:** Tools like **Amundsen** and **DataHub** can auto-ingest metadata using connectors or APIs.

Data governance ensures that data is accurate, available, and secure across its lifecycle. It includes policies, processes, and roles that define how data is managed and accessed.

- **Core Governance Functions:**
 - **Ownership tracking** (Who owns the data?) ○ **Lineage tracking** (Where did the data come from?) ○ **Schema evolution** (How does structure change over time?)
 - **Security and access control** (Who can see/edit the data?)

Governance ensures compliance, data quality, and risk reduction — critical for enterprise-level systems.

Data sharing allows internal or external users to access datasets securely via web interfaces or APIs. This improves collaboration, transparency, and reusability of data across departments or institutions.

- **Modes of Sharing:**
 - Public sharing (e.g., open government data)
 - Internal controlled access (role-based or authenticated access)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

- **Tools Used:** Platforms like **CKAN** allow you to publish and manage datasets with customizable metadata and licenses.

=>Tools Used

Amundsen (by Lyft) – Metadata Cataloging Tool

Amundsen is an open-source data discovery and metadata platform designed to improve productivity and data trust.

- **Key Components:**
 - **Frontend:** React-based web UI for searching and exploring datasets.
 - **Backend:**
 - Metadata stored in **Neo4j** (or alternatives like RDS).
 - Search powered by **Elasticsearch**.
 - **Ingestion:** Uses **Amundsen Data Builder** to pull metadata from data sources like PostgreSQL, MySQL, Hive, dbt, and Airflow.
- **Features:**
 - Dataset search and preview ○ Owner and tag display
 - Lineage visualization (when manually or programmatically integrated)

CKAN (Comprehensive Knowledge Archive Network) – Data Sharing Portal

CKAN is an open-source data management system for powering data hubs and open data portals.

- **Core Features:**
 - Dataset publishing with full metadata ○ Support for data previews, file uploads, APIs ○ License tagging and access control ○ Plugin architecture for customization
 - Organization-level dataset management
- **Ideal Use Case:** Public sector open data portals, research data repositories.



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

Lineage Tracking

Understanding how data flows across pipelines and transformations is essential for trust and troubleshooting. Tools like:

- **DataHub**: Native integration with dbt, Airflow, and Spark to visualize lineage.
- **Amundsen**: Lineage support exists, but often requires manual or custom integration scripts.

User Access and Security

- **Authentication**: OAuth2 or LDAP-based identity management.
- **Authorization**: Role-based access control (RBAC) using proxies like **Apache Ranger**.
- **API Security**: Use API gateways like **Kong** or **Traefik** to secure metadata endpoints.

Scheduling Metadata Updates

Regular metadata updates ensure your catalog remains accurate. Use:

- **Apache Airflow / Prefect**: Workflow orchestrators to schedule metadata ingestion jobs.
- **Cron Jobs**: Simple scheduling mechanism to trigger datahub ingest or Amundsen jobs periodically.

Output:

Tools: Amundsen & CKAN

Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
competent_kilby	e490e5b1adb	maradb:10.5		0%	1 day ago	[Stop] [Restart] [Logs]
amundsen	-	-	-	0%	51 minutes ago	[Stop] [Restart] [Logs]
docker	-	-	-	3.84%	3 hours ago	[Stop] [Restart] [Logs]
ckan	-	-	-	1.14%	17 minutes ago	[Stop] [Restart] [Logs]
cloud-vault	-	-	-	0.04%	3 hours ago	[Stop] [Restart] [Logs]



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

1: Deploy a Metadata Catalog

amundsen.py

```
import pandas as pd import json import
requests from neo4j import
GraphDatabase import os import logging
import time from elasticsearch import
Elasticsearch

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(name)s
-
%(levelname)s - %(message)s') logger
= logging.getLogger(__name__)

# Neo4j connection details
NEO4J_URI = "bolt://localhost:7687"
NEO4J_USER = "neo4j"
NEO4J_PASSWORD = "test" # Change to your Neo4j password

# Elasticsearch connection details
ES_HOST = "localhost" ES_PORT
= 9200

# Amundsen API (frontend) details
AMUNDSEN_API = "http://localhost:5000/api"

# CSV file path
CSV_FILE = "MOCK_DATA.csv"
def
load_and_analyze_csv():
    """Load CSV file and analyze its structure""" logger.info(f>Loading
    CSV file: {CSV_FILE}")
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
try:
    # Read the CSV file
    df = pd.read_csv(CSV_FILE)

    # Basic analysis
    row_count = len(df)
    column_count = len(df.columns)
    columns = df.columns.tolist()

    # Infer column types
    column_types = {}
    for col in columns:
        if df[col].dtype == 'int64':
            column_types[col] = 'integer'
        elif df[col].dtype == 'float64':
            column_types[col] = 'float'
        else:
            column_types[col] = 'string'

    # Data quality checks
    null_counts = df.isnull().sum().to_dict()
    unique_counts = {col: df[col].nunique() for col in columns}

    logger.info(f"CSV Analysis Complete: {row_count} rows, {column_count}
columns")

    return {
        'file_name': CSV_FILE,
        'row_count': row_count,
        'column_count': column_count,
        'columns': columns,
        'column_types': column_types,
        'null_counts': null_counts,
        'unique_counts': unique_counts
    }
except Exception as e:
    logger.error(f"Failed to load CSV: {e}")
    raise

def create_table_metadata_in_neo4j(csv_analysis):
    """Insert table and column metadata directly into Neo4j"""
    logger.info("Creating table metadata in Neo4j")

    try:
        # Connect to Neo4j
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

        # Create database, cluster, schema nodes if they don't exist
        with driver.session() as session:
            # Create database node
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
session.run("""
MERGE (db:Database {key: $db_key})
SET db.name = $db_name
""", db_key="csv_database", db_name="CSV Database")

# Create cluster node
session.run("""
MATCH (db:Database {key: $db_key})
MERGE (cluster:Cluster {key: $cluster_key})
SET cluster.name = $cluster_name
MERGE (cluster)-[:CLUSTER_OF]->(db)
""", db_key="csv_database", cluster_key="csv_database://csv_cluster",
cluster_name="CSV Cluster")

# Create schema node
session.run("""
MATCH (cluster:Cluster {key: $cluster_key})
MERGE (schema:Schema {key: $schema_key})
SET schema.name = $schema_name
MERGE (schema)-[:SCHEMA_OF]->(cluster)
""", cluster_key="csv_database://csv_cluster",
schema_key="csv_database://csv_cluster.public", schema_name="public")

# Create table node table_key =
"csv_database://csv_cluster.public/user_data"
session.run("""
MATCH (schema:Schema {key: $schema_key})
MERGE (table:Table {key: $table_key})
SET table.name = $table_name,
table.schema = $schema_name,
table.cluster = $cluster_name,
table.database = $db_name,
table.description = $description,
table.last_updated_epoch = $last_updated_epoch
MERGE (table)-[:TABLE_OF]->(schema)
""",
schema_key="csv_database://csv_cluster.public",
, table_key=table_key, table_name="user_data",
schema_name="public",
cluster_name="CSV Cluster", # Use consistent capitalization
db_name="CSV Database", # Use consistent naming
description=f"User data loaded from CSV with
{csv_analysis['row_count']} rows and {csv_analysis['column_count']} columns",
last_updated_epoch=int(pd.Timestamp.now().timestamp()))

# Create column nodes for i, col_name in
enumerate(csv_analysis['columns']):
col_type = csv_analysis['column_types'].get(col_name, 'string')
null_count = csv_analysis['null_counts'].get(col_name, 0)
unique_count = csv_analysis['unique_counts'].get(col_name, 0)
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
completeness = ((csv_analysis['row_count'] - null_count) /
csv_analysis['row_count']) * 100 if csv_analysis['row_count'] > 0 else 100
uniqueness = (unique_count / csv_analysis['row_count']) * 100 if
csv_analysis['row_count'] > 0 else 100

description = f"Column {col_name}. Data quality:
{completeness:.1f}% complete, {uniqueness:.1f}% unique values."

col_key = f"{table_key}/{col_name}"
session.run("""
MATCH (table:Table {key: $table_key})
MERGE (column:Column {key: $col_key})
SET column.name = $col_name,
    column.type = $col_type,
    column.description = $description,
    column.sort_order = $sort_order
MERGE (column)-[:COLUMN_OF]->(table)
""",
    table_key=table_key,
    col_key=col_key,
    col_name=col_name,
    col_type=col_type,
    description=description,
    sort_order=i)

# Close Neo4j connection
driver.close()
logger.info("Table metadata created in Neo4j")
except Exception as e:
    logger.error(f"Failed to create Neo4j metadata: {e}")
    raise

def create_lineage_in_neo4j():
    """Create data lineage in Neo4j"""
    logger.info("Creating data lineage in Neo4j")

    try:
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

        with driver.session() as session:
            # Create source table (raw data)
            session.run("""
            MATCH (db:Database {key: "csv_database"})
            MATCH (cluster:Cluster {key: "csv_database://csv_cluster"})
            MERGE (schema:Schema {key: "csv_database://csv_cluster.raw"})
            SET schema.name = "raw"
            MERGE (schema)-[:SCHEMA_OF]->(cluster)

            MERGE (table:Table {key:
"csv_database://csv_cluster.raw/raw_user_data"})
            SET table.name = "raw_user_data",
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
        table.schema = "raw",
        table.cluster = "CSV Cluster",
        table.database = "CSV Database",
        table.description = "Raw user data imported from external system"
MERGE (table)-[:TABLE_OF]->(schema)
""")

# Create analytics table (derived data)
session.run("""
MATCH (db:Database {key: "csv_database"})
MATCH (cluster:Cluster {key: "csv_database://csv_cluster"})
MERGE (schema:Schema {key: "csv_database://csv_cluster.analytics"})
SET schema.name = "analytics"
MERGE (schema)-[:SCHEMA_OF]->(cluster)

MERGE (table:Table {key:
"csv_database://csv_cluster.analytics/user_location_analytics"})
SET table.name = "user_location_analytics",
    table.schema = "analytics",
    table.cluster = "CSV Cluster",
    table.database = "CSV Database",
    table.description = "Analytics table derived from user data with
IP-based location analysis"
MERGE (table)-[:TABLE_OF]->(schema)
""")

# Create lineage relationships
session.run("""
MATCH (source:Table {key:
"csv_database://csv_cluster.raw/raw_user_data"})
MATCH (target:Table {key:
"csv_database://csv_cluster.public/user_data"})
MERGE (target)-[:DOWNSTREAM_OF {source: "ETL_PROCESS"}]->(source)
""")

session.run("""
MATCH (source:Table {key:
"csv_database://csv_cluster.public/user_data"})
MATCH (target:Table {key:
"csv_database://csv_cluster.analytics/user_location_analytics"})
MERGE (target)-[:DOWNSTREAM_OF {source: "ANALYTICS_PIPELINE"}]-
>(source)
""")

driver.close()
logger.info("Data lineage created in Neo4j")
except Exception as e:
    logger.error(f"Failed to create lineage: {e}")
    raise

def add_table_owner():
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
"""Add table ownership information"""
logger.info("Adding table ownership information")

try:
    driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

    with driver.session() as session:
        # Create user node
        session.run("""
        MERGE (user:User {key: "user@example.com"})
        SET user.email = "user@example.com",
            user.first_name = "Test",
            user.last_name = "User",
            user.full_name = "Test User",
            user.github_username = "testuser",
            user.team_name = "Data Engineering",
            user.employee_type = "FTE"
        """)

        # Create owner relationship
        session.run("""
        MATCH (table:Table {key:
"csv_database://csv_cluster.public/user_data"})
        MATCH (user:User {key: "user@example.com"})
        MERGE (table)-[:OWNER]->(user)
        """)

        driver.close()
        logger.info("Table ownership added")
except Exception as e:
    logger.error(f"Failed to add ownership: {e}")
    raise


def add_table_tags():
    """Add tags to the table"""
    logger.info("Adding tags to the table")

    try:
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))

        with driver.session() as session:
            # Create tag nodes
            tags = ["user_data", "personal_info", "experiment"]
            for tag in tags:
                session.run("""
                MERGE (tag:Tag {key: $tag_key})
                SET tag.tag_type = "default"
                """, tag_key=tag)

            # Link tags to table
            session.run("""
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
MATCH (table:Table {key:
"csv_database://csv_cluster.public/user_data"})
MATCH (tag:Tag {key: $tag_key})
MERGE (table)-[:TAGGED_BY]->(tag)
"", tag_key=tag)

driver.close()
logger.info("Tags added to table")
except Exception as e:
    logger.error(f"Failed to add tags: {e}")
    raise

def index_data_in_elasticsearch():
    """Create Elasticsearch indices required by Amundsen"""
    logger.info("Indexing data in Elasticsearch")

    try:
        # Connect to Elasticsearch
        es = Elasticsearch([{'host': ES_HOST, 'port': ES_PORT}])

        # Verify connection
        if not es.ping():
            logger.error("Could not connect to Elasticsearch")
            return False

        # Check if indices exist
        indices = ["table_search_index", "user_search_index",
"dashboard_search_index"]
        missing_indices = []

        for index in indices:
            if not es.indices.exists(index=index):
                missing_indices.append(index)

        # Create index with basic mapping
        if index == "table_search_index":
            mapping = {
                "mappings": {
                    "properties": {
                        "name": {"type": "text"},
                        "schema": {"type": "keyword"},
                        "display_name": {"type": "keyword"},
                        "description": {"type": "text"},
                        "column_names": {"type": "text"},
                        "column_descriptions": {"type": "text"},
                        "tags": {"type": "keyword"},
                        "cluster": {"type": "keyword"},
                        "database": {"type": "keyword"},
                        "key": {"type": "keyword"}
                    }
                }
            }
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
}
es.indices.create(index=index, body=mapping)

# Index table data directly
table_doc = {
    "name": "user_data",
    "key": "csv_database://csv_cluster.public/user_data",
    "display_name": "user_data",
    "schema": "public",
    "cluster": "CSV Cluster",
    "database": "CSV Database",
    "description": "User data loaded from CSV",
    "column_names": ["id", "first_name", "last_name", "email", "gender",
"ip_address"],
    "column_descriptions": ["User ID", "First Name", "Last Name", "Email
Address", "Gender", "IP Address"],
    "tags": ["user_data", "personal_info", "experiment"],
    "last_updated_timestamp": int(time.time())
}

# Index document
es.index(index="table_search_index",
id="csv_database://csv_cluster.public/user_data", body=table_doc)

# Force refresh to make document immediately available for search
es.indices.refresh(index="table_search_index")

logger.info("Data indexed in Elasticsearch")
return True
except Exception as e:
    logger.error(f"Failed to index in Elasticsearch: {e}")
    return False

def verify_amundsen_status():
    """Check if Amundsen frontend is accessible"""
    try:
        response = requests.get(f"{AMUNDSEN_API}/healthcheck")
        if response.status_code == 200:
            logger.info("Amundsen frontend is accessible")
            return True
        else:
            logger.warning(f"Amundsen returned status code:
{response.status_code}")
            return False
    except requests.exceptions.ConnectionError:
        logger.warning("Could not connect to Amundsen frontend")
        return False

def verify_data_in_amundsen():
    """Verify data is searchable in Amundsen"""
    try:
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
# Search for table
search_url = f"{AMUNDSEN_API}/search/v0/search"
search_params = {
    "query": "user_data",
    "resource": "table",
    "index": 0,
    "page_index": 0
}

response = requests.get(search_url, params=search_params)

if response.status_code == 200:
    results = response.json()
    if results.get('results'):
        logger.info("Data found in Amundsen search")
        return True
    else:
        logger.warning("No data found in Amundsen search")
        return False
else:
    logger.warning(f"Search API returned status code: {response.status_code}")
    return False
except requests.exceptions.ConnectionError:
    logger.warning("Could not connect to Amundsen search API")
    return False
except Exception as e:
    logger.error(f"Error verifying data in Amundsen: {e}")
    return False

def run_pipeline():
    """Run the full ingestion pipeline"""
    try:
        # Step 1: Analyze CSV
        csv_analysis = load_and_analyze_csv()

        # Step 2: Create table metadata in Neo4j
        create_table_metadata_in_neo4j(csv_analysis)

        # Step 3: Create data lineage
        create_lineage_in_neo4j()

        # Step 4: Add table owner
        add_table_owner()

        # Step 5: Add tags
        add_table_tags()

        # Step 6: Index data in Elasticsearch
        es_indexed = index_data_in_elasticsearch()
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
# Step 7: Verify Amundsen status
amundsen_accessible = verify_amundsen_status()

# Step 8: Verify data in Amundsen
if amundsen_accessible:
    data_visible = verify_data_in_amundsen()
else:
    data_visible = False

logger.info("Pipeline execution summary:")
logger.info(f" - Data loaded to Neo4j: Success")
logger.info(f" - Data indexed in Elasticsearch: {'Success' if es_indexed
else 'Failed'}")
logger.info(f" - Amundsen frontend accessible: {'Yes' if
amundsen_accessible else 'No'}")
logger.info(f" - Data visible in Amundsen: {'Yes' if data_visible else
'No'}")

if not es_indexed:
    logger.warning("⚠ Elasticsearch indexing failed - data won't be
searchable")
    logger.warning("Check if Elasticsearch is running with: curl
http://localhost:9200")

if not amundsen_accessible:
    logger.warning("⚠ Amundsen frontend not accessible - check if it's
running")
    logger.warning("Verify with: docker ps | grep amundsen")

if not data_visible and amundsen_accessible:
    logger.warning("⚠ Data not visible in Amundsen even though frontend is
accessible")
    logger.warning("Check Elasticsearch indices and Amundsen frontend
logs")

logger.info("You can view the data in Amundsen at http://localhost:5000")

# Return success flag for troubleshooting
return es_indexed and amundsen_accessible and data_visible

except Exception as e:
    logger.error(f"Pipeline failed: {e}")
    return False

def verify_neo4j_data():
    """Verify data in Neo4j - run this to troubleshoot if data is not showing"""
    logger.info("Verifying Neo4j data...")

    try:
        driver = GraphDatabase.driver(NEO4J_URI, auth=(NEO4J_USER, NEO4J_PASSWORD))
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
with driver.session() as session:
    # Check table
    table_result = session.run("""
MATCH (t:Table {name: 'user_data'}) RETURN t
""")

    table_data = [record["t"] for record in table_result]
    if table_data:
        logger.info("✅ Table 'user_data' found in Neo4j")
    else:
        logger.warning("❌ Table 'user_data' NOT found in Neo4j")

    # Check columns
    col_result = session.run("""
MATCH (c:Column)-[:COLUMN_OF]->(t:Table {name: 'user_data'})
RETURN c.name as column_name
""")

    columns = [record["column_name"] for record in col_result]
    if columns:
        logger.info(f"✅ Found {len(columns)} columns for table 'user_data': {columns}")
    else:
        logger.warning("❌ No columns found for table 'user_data'")

    # Check lineage
    lineage_result = session.run("""
MATCH (t1:Table)-[:DOWNSTREAM_OF]->(t2:Table)
WHERE t1.name = 'user_data' OR t2.name = 'user_data'
RETURN t1.name as target, t2.name as source
""")

    lineage = [(record["source"], record["target"]) for record in lineage_result]
    if lineage:
        logger.info(f"✅ Found lineage relationships: {lineage}")
    else:
        logger.warning("❌ No lineage relationships found")

    driver.close()
except Exception as e:
    logger.error(f"Failed to verify Neo4j data: {e}")

def verify_elasticsearch_indices():
    """Verify Elasticsearch indices required by Amundsen"""
    logger.info("Verifying Elasticsearch indices...")

    try:
        # Connect to Elasticsearch
        es = Elasticsearch([{'host': ES_HOST, 'port': ES_PORT}])
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
if not es.ping():
    logger.error("✗ Could not connect to Elasticsearch")
    return

logger.info("☑ Connected to Elasticsearch")

# Check indices
indices = ["table_search_index", "user_search_index",
"dashboard_search_index"]
for index in indices:
    if es.indices.exists(index=index):
        logger.info(f"☑ Index '{index}' exists")

        # Check document count
        count = es.count(index=index)
        logger.info(f" - Document count: {count['count']}")

        # If table index, check for our document
        if index == "table_search_index":
            try:
                doc = es.get(index=index,
id="csv_database://csv_cluster.public/user_data")
                if doc['found']:
                    logger.info(f"☑ Table document found in index")
                else:
                    logger.warning(f"✗ Table document NOT found in index")
            except:
                logger.warning(f"✗ Table document NOT found in index")
        else:
            logger.warning(f"✗ Index '{index}' does NOT exist")

except Exception as e:
    logger.error(f"Failed to verify Elasticsearch indices: {e}")

if __name__ == "__main__":
    print("=" * 80)
    print("AMUNDSEN DATA INGESTION AND TROUBLESHOOTING SCRIPT")
    print("=" * 80)
    print("\nThis script will:")
    print("1. Load and analyze your CSV data")
    print("2. Create metadata in Neo4j")
    print("3. Index data in Elasticsearch")
    print("4. Verify everything is working")
    print("\nChoose an option:")
    print("1. Run full pipeline")
    print("2. Verify Neo4j data only")
    print("3. Verify Elasticsearch indices only")
    print("4. Run full pipeline with verification")

    try:
        choice = input("\nEnter your choice (1-4): ")
```



Shri Vile Parle Kelavani Mandal's

DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING

(Autonomous College Affiliated to the University of Mumbai)

NAAC Accredited with "A" Grade (CGPA : 3.18)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

```
if choice == '1':
    run_pipeline()
elif choice == '2':
    verify_neo4j_data()
elif choice == '3':
    verify_elasticsearch_indices()
elif choice == '4':
    success = run_pipeline()
    if not success:
        print("\nTroubleshooting:")
        verify_neo4j_data()
        verify_elasticsearch_indices()
else:
    print("Invalid choice. Please run again.")
except KeyboardInterrupt:
    print("\nScript interrupted by user.")
except Exception as e:
    print(f"\nAn error occurred: {e}")
```

Metadata Ingestion Pipeline

```
(myenv) PS E:\Migration\Temp\DE_expt> python .\csv_loader.py

=====
AMUNDSEN TAG FIX UTILITY
=====

This script will fix tag relationships in Neo4j to work properly with Amundsen without Elasticsearch

Choose an option:
1. Fix tag relationships
2. Add Amundsen-specific metadata
3. Verify tag setup
4. Run all fixes
5. Create test data (if none exists)

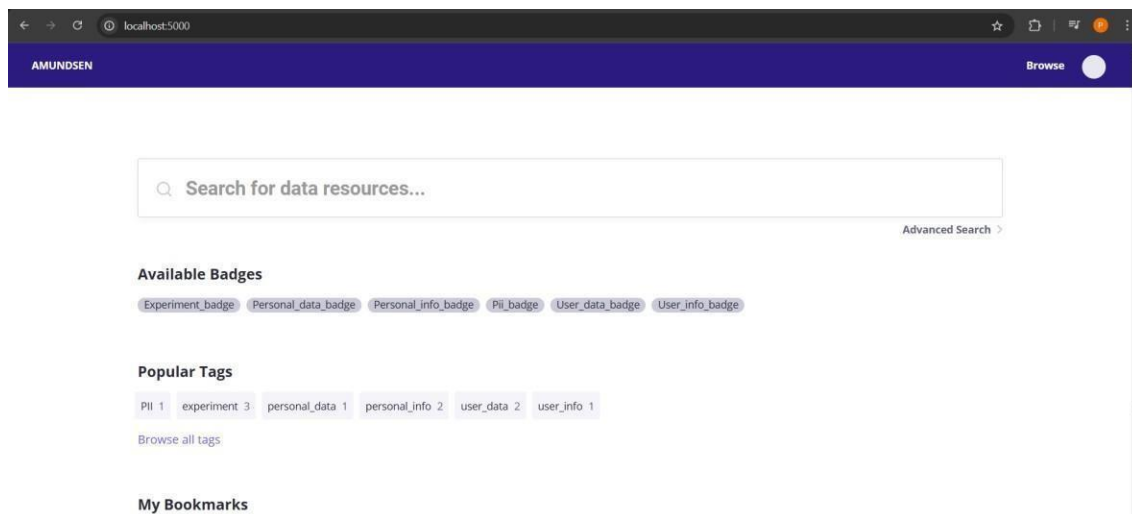
Enter your choice (1-5): 4
2025-05-05 23:12:56,415 - __main__ - INFO - Fixing tag relationships for Amundsen...
2025-05-05 23:12:56,416 - __main__ - INFO - Cleaning up duplicate tags...
2025-05-05 23:12:56,800 - __main__ - INFO - Updating tag properties...
2025-05-05 23:12:56,898 - __main__ - INFO - Creating bidirectional tag relationships...
2025-05-05 23:12:57,310 - __main__ - INFO - Creating tag search metadata...
2025-05-05 23:12:57,397 - __main__ - INFO - Creating lookup paths for Amundsen...
2025-05-05 23:12:57,471 - __main__ - INFO - Adding tag usage counters...
2025-05-05 23:12:57,558 - __main__ - INFO - Verifying tag setup...
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'personal_info' is connected to 2 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'experiment' is connected to 3 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'user_data' is connected to 2 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'PII' is connected to 1 tables
2025-05-05 23:12:57,707 - __main__ - INFO - Tag 'personal_data' is connected to 1 tables
2025-05-05 23:12:57,708 - __main__ - INFO - Tag 'user_info' is connected to 1 tables
2025-05-05 23:12:57,708 - __main__ - INFO - Tags fixed successfully
2025-05-05 23:12:57,708 - __main__ - INFO - Adding Amundsen-specific metadata...
2025-05-05 23:12:58,663 - __main__ - INFO - Amundsen-specific metadata added
2025-05-05 23:12:58,663 - __main__ - INFO - Verifying tag setup for Amundsen...
2025-05-05 23:12:58,804 - __main__ - INFO - Found table: user_data
2025-05-05 23:12:58,828 - __main__ - INFO - Found 6 tags
2025-05-05 23:12:58,877 - __main__ - INFO - Found 10 tag-to-table relationships
2025-05-05 23:12:58,877 - __main__ - INFO - Tag 'personal_info' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,877 - __main__ - INFO - Tag 'personal_info' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'experiment' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'experiment' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'experiment' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,878 - __main__ - INFO - Tag 'user_data' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'user_data' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'PII' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'personal_data' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,879 - __main__ - INFO - Tag 'user_info' -> 'TAG_OF' -> Table 'user_data'
2025-05-05 23:12:58,911 - __main__ - INFO - Found 6 search result nodes for tags
2025-05-05 23:12:58,976 - __main__ - INFO - Found 1 tables with display names for Amundsen
2025-05-05 23:12:59,012 - __main__ - INFO - Found 6 tag dashboard connections
2025-05-05 23:12:59,060 - __main__ - INFO - Found 6 tag badge connections
2025-05-05 23:12:59,105 - __main__ - INFO - Found 10 TAGGED relationships from tags to tables
2025-05-05 23:12:59,158 - __main__ - INFO - Found 6 tags with proper Amundsen properties
```



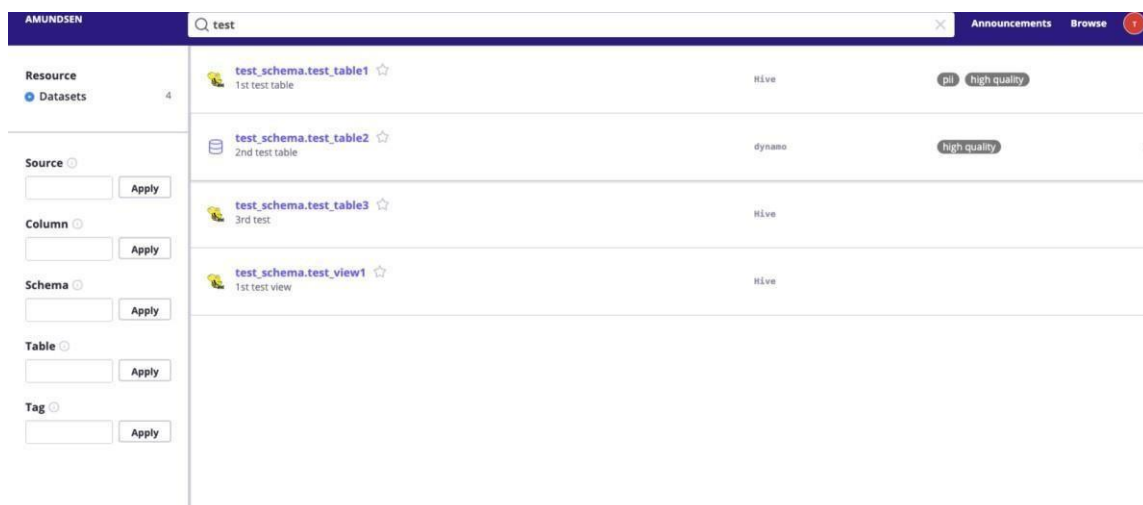
Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

localhost:5000

Metatags Personal_Info, User_Info, Experiment_Badge



DATABASES



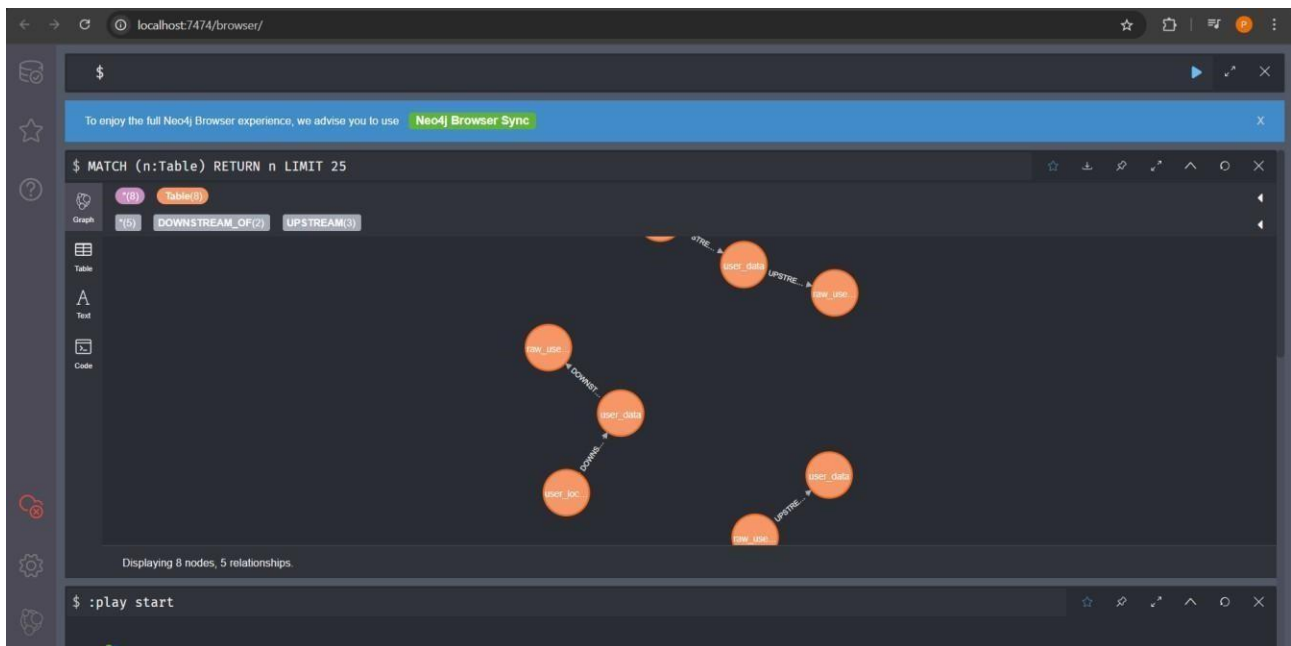


Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

DATASET Values

default.event_amundsenfrontend_user_action						
	USER_VALUE	POS_ARGS_JSON	START_EPOCH_MS	HTTP_REQUEST_ID	ERROR	PRODUCER_SERVICE
	dmittow@lyft.com	[]	1593536550891	635a2231-8c06-49e5-9e60-74b643f58a3e	null	amundsenfront
	alewis@lyft.com	[]	1593507028600	c848704-72d1-4c09-bb6a-c08a509000	null	amundsenfront
vive/default/event_pricing_data_worker_executed", "value": null]	jamiekang@lyft.com	[]	1593539861251	51c0364e-34ee-4c80-bb09-ae9c7c13e9e	null	amundsenfront
	johnchen@lyft.com	[]	1593539692208	01d58694-42e4-44b6-8fcc-56929cd46214	null	amundsenfront
	cindyzhang@lyft.com	[]	1593536621998	c7b394a-c1ea-4c62-be82-d0c3c8b5530	null	amundsenfront
	mingyangli@lyft.com	[]	1593539619093	630cc144-bc2d-4960-a7cb-4fb2486709d	null	amundsenfront
"location": "search", "value": null]	johnchen@lyft.com	[]	1593538012325	3617e16b-bbe4-45e9-8e56-2d9ba0d3f34b	null	amundsenfront
hive/core/fact_rides", "value": null]	plipkask@lyft.com	[]	1593537176335	1e80925d-6400-4bdc-84bd-139863d52596	null	amundsenfront
	ksineen@lyft.com	[]	1593538058370	3d7cde4-fa34-4524-82ac-f80fa26fa188	null	amundsenfront
	ksineen@lyft.com	[]	1593538038768	661e9bb7-9a19-49ae-8a70-ccdc3f9c2bb9	null	amundsenfront
	ddobson@lyft.com	[]	1593539378651	541b645b-1a5b-4c0c-a204-6e084c9f331	null	amundsenfront
	mingyangli@lyft.com	[]	1593539609464	e142d6d9-a51c-4994-9164-27d92ac435d4	null	amundsenfront
	ckiszek@lyft.com	[]	1593539769808	abf697f1-644c-4d48-89d7-49drc8f65a0	null	amundsenfront
	ccchung@lyft.com	[]	1593537593829	58416d3-7d07-455b-a85e-609596b612bf	null	amundsenfront
	delahstane@lyft.com	[]	1593536573215	0d4f00c5-929b-4ca1-8a9c-3da0f5734d086	null	amundsenfront
in": null, "bounced_code": null, "bounced_description": null, "dropped_reason": null, "dropped_code": null, "clicked_url": null]	mailer.mailer@lyft.com	[]	1593536425497	7853d247-a0e2-4091-b092-81a076c1afdd	null	amundsenfront
	cindyzhang@lyft.com	[]	1593536621999	b30a6798-843e-4543-a40c-e6bd8bc0be10	null	amundsenfront
	esagay@lyft.com	[]	1593537727233	e2046500-a477-4544-807b-ccc40e6d2e19	null	amundsenfront
	csun@lyft.com	[]	1593539926096	57a7a402-2657-4c0d-91c2-22755f704652	null	amundsenfront
	lukezhang@lyft.com	[]	1593537620277	5730a168-8395-4531-9db6-2b49a3978881	null	amundsenfront
	dsalman@lyft.com	[]	159353791060	9f5e998f-d933-4871-b79f-fe899d9418d2	null	amundsenfront

Neo4j Data lineage

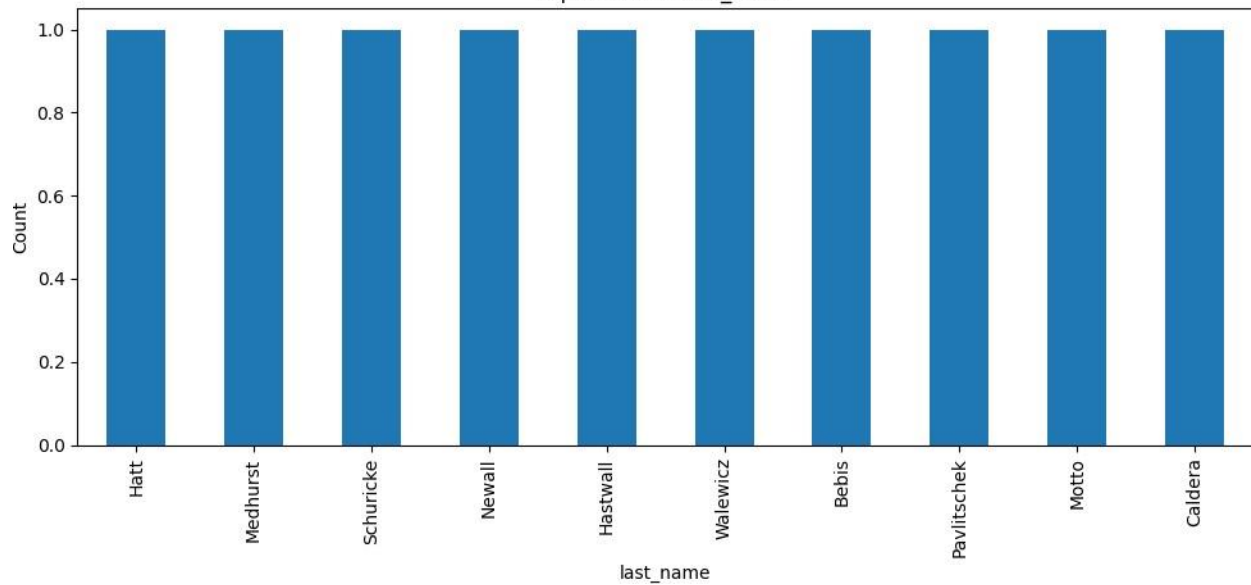


Lineage Tracking



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

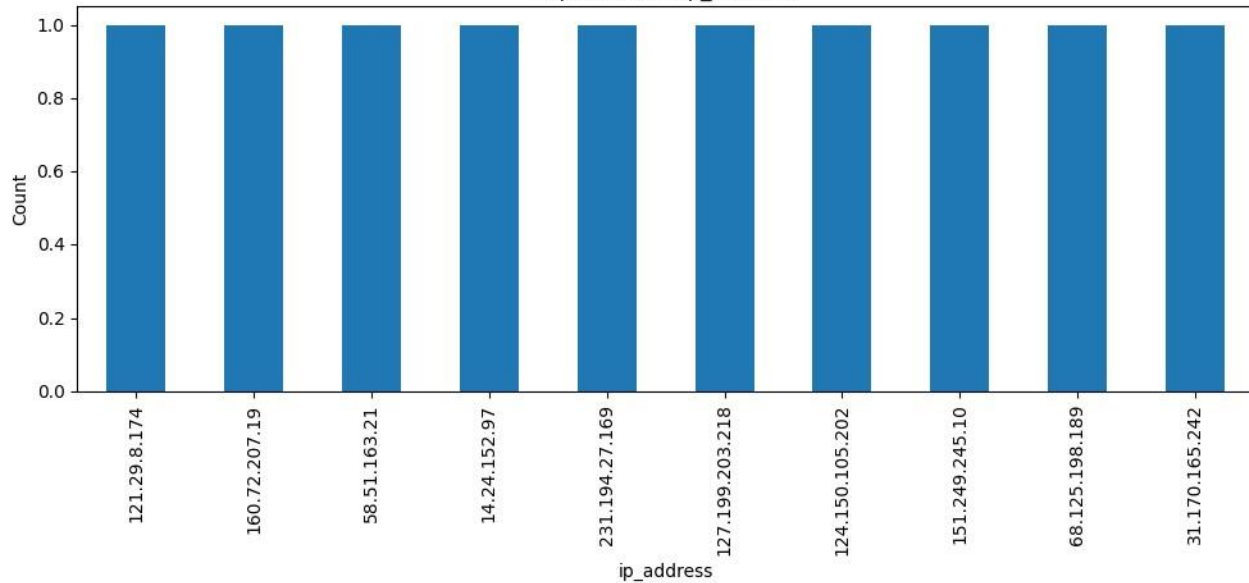
Top values in last_name



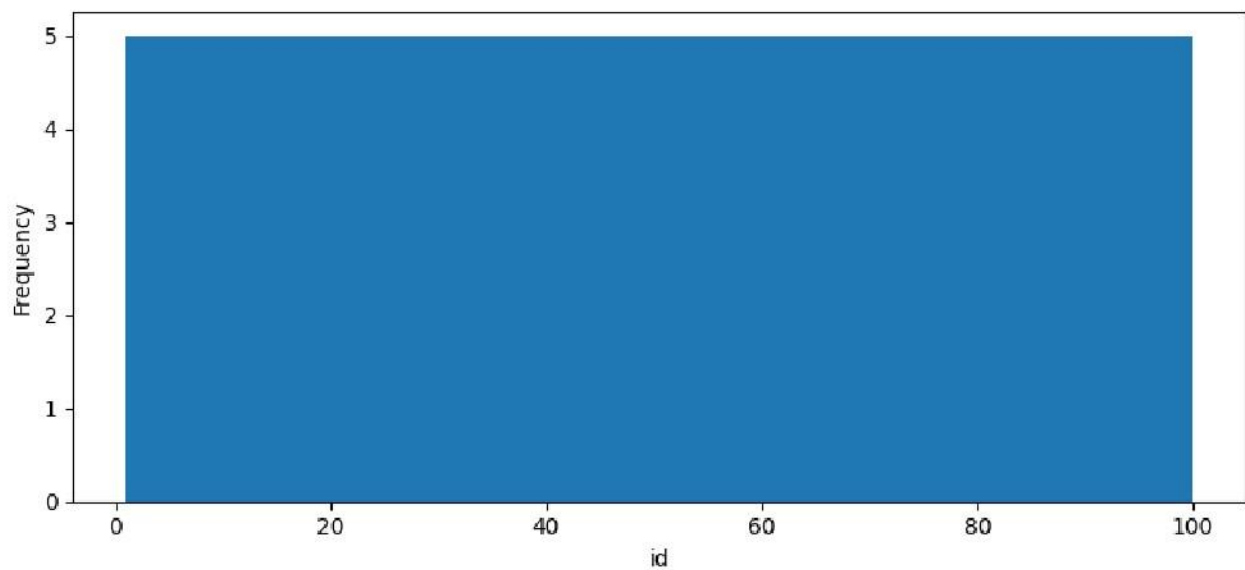


Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

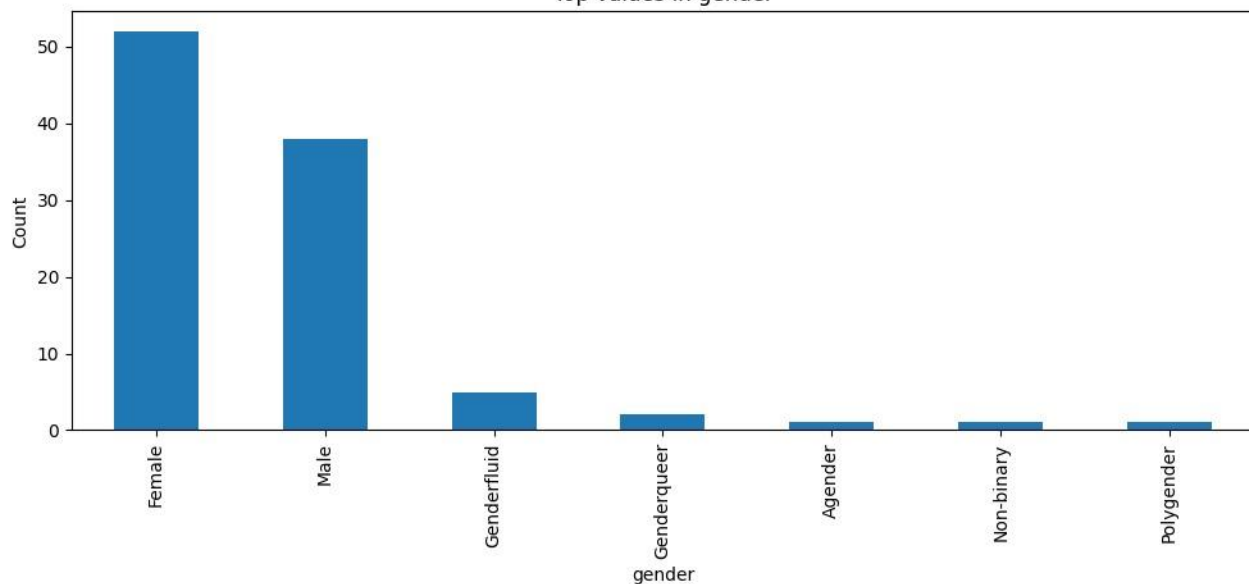
Top values in ip_address



Distribution of id

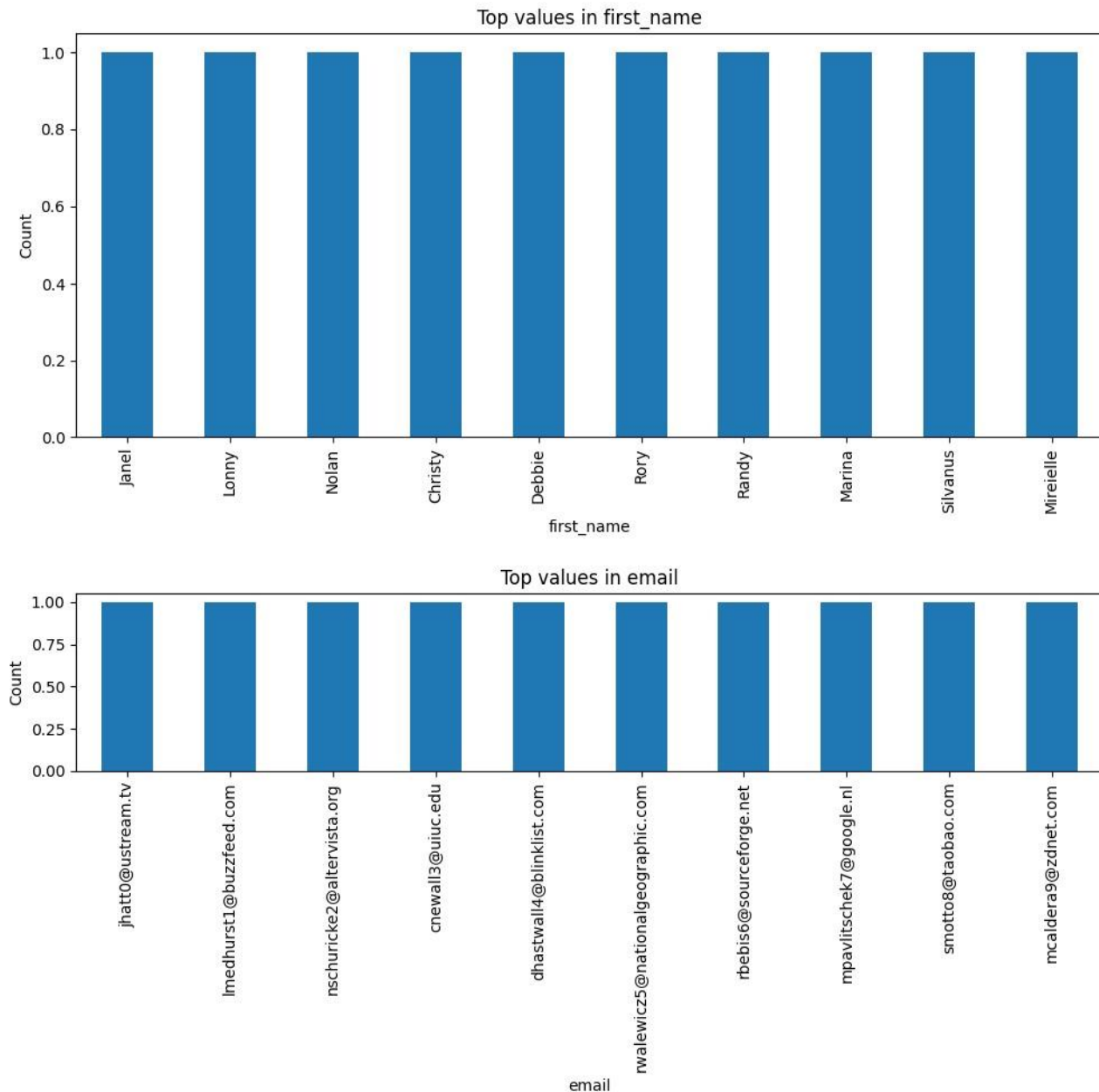


Top values in gender





Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)



Data Sharing Portal

```
brett@docker:~$ sudo docker container ls
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS                PORTS
782aa24b8447   ckan-docker-ckan                    "/srv/app/start_ckan..." 3 minutes ago  Up 2 minutes (health: starting)  0.0.0.0:50
046d6c221765   ckan/ckan-solr:2.9-solr8            "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes (healthy)          8983/tcp
46f557482958   ckan/ckan-base-datapusher:0.0.19    "sh -c 'uwsgi --plug..." 3 minutes ago  Up 3 minutes (healthy)          8800/tcp
er
3a0935bc4dee   redis:6                             "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes (healthy)          6379/tcp
c63f1b04f5c7   ckan-docker-db                      "docker-entrypoint.s..." 3 minutes ago  Up 3 minutes (healthy)          5432/tcp
brett@docker:~$
brett@docker:~$ sudo docker container ls
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS                PORTS
782aa24b8447   ckan-docker-ckan                    "/srv/app/start_ckan..." 4 minutes ago  Up 2 minutes (health: starting)  0.0.0.0:50
046d6c221765   ckan/ckan-solr:2.9-solr8            "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)          8983/tcp
46f557482958   ckan/ckan-base-datapusher:0.0.19    "sh -c 'uwsgi --plug..." 4 minutes ago  Up 4 minutes (healthy)          8800/tcp
er
3a0935bc4dee   redis:6                             "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)          6379/tcp
c63f1b04f5c7   ckan-docker-db                      "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes (healthy)          5432/tcp
```



Department of Computer Science and Engineering (IoT and Cyber Security with Block Chain Technology)

The image displays two screenshots of the CKAN web interface. The top screenshot shows the home page with a 'Welcome to CKAN' message, a search bar, and a 'Popular tags' section. The bottom screenshot shows the user profile for 'ckan_admin', which includes a profile picture, a bio, and statistics for followers and datasets.

Home Page:

- Header: ckan logo, navigation links (Datasets, Organizations, Groups, About), and a search bar.
- Search bar: "Search data" with a placeholder "E.g. environment".
- Popular tags: A section for popular tags.
- Footer: Links for "About CKAN", "CKAN API", and "CKAN Association".

User Profile (ckan_admin):

- Header: ckan logo, navigation links (Datasets, Organizations, Groups, About).
- Profile picture: A purple geometric logo.
- Username: ckan_admin
- Bio: You have not provided a biography.
- Followers: 0
- Datasets: 0
- Username: ckan_admin
- Email Private: your_email@example.com
- Member Since: [Not specified]

Conclusion: In conclusion, the implementation showed data cataloging and sharing services using open-source tools like Amundsen and CKAN. The setup enabled automated metadata ingestion, governance, and secure dataset sharing, demonstrating practical applications of modern data management frameworks